Homework 1 – Version A: My Grade Average

Course: Mobile Application and Service Development **Level:** basic · **Points:** 7 · **Deadline:** 31.10.2026 at 23:59 · **Defense:** lab session, 02.11.2026

Description

You write a small Kotlin program that keeps a record of one student's courses and prints a report on their studies. You design the data model yourself. At the defense you will justify your model.

Data

Enter this data in your code, using the model you designed.

Code	Course	Semester	ECTS	Type	Grade
MAT1	Mathematics 1	1	7	mandatory	8
FIZ1	Physics 1	1	6	mandatory	not passed
ESKE001	Fundamentals of Electrical Engineering 1	1	7	mandatory	7
RI001	Fundamentals of Computing	1	5	mandatory	10
MAT2	Mathematics 2	2	7	mandatory	not passed
ESKE002	Fundamentals of Electrical Engineering 2	2	7	mandatory	6
RI101	Fundamentals of Programming	2	6	mandatory	10
RI202	Object-Oriented Programming	3	6	mandatory	not passed
BMI203	Medical Image Processing and Analysis	3	6	elective	9
BMI201	Clinical Engineering	3	6	elective	not passed

Model requirements

Your model must satisfy the following:

1. Two courses with the same data must be equal when compared with `==`, and printing a course with `println` must be readable, without writing that logic by hand.
2. Course data cannot be changed after the course is created.
3. If you misspell the course type in your code (e.g. "mandatroy" instead of "mandatory"), the program must not even compile.

4. A course that is not passed has no grade. If you try to calculate with a grade (e.g. add it up or compute an average) without checking that it exists, the program must not compile.

What the program must calculate

1. How many courses are passed, out of the total number.
2. The grade average, for passed courses only.
3. The ECTS-weighted grade average, for passed courses only: each grade is multiplied by the course's ECTS credits, and the sum is divided by the total ECTS credits of passed courses.
4. The total number of ECTS credits earned, for passed courses only.
5. The names of all courses not passed.
6. The course with the best grade. If there are several, take the first one in the list.
7. Write a function that returns all courses from a given semester. In `main`, call it for **semester 3** and print each course on its own line, in the format shown in the expected output.

Code rules

- Queries over the list are done with **collection functions** (`filter`, `map`, `count`...), not `for`/`while` loops. You may use a loop or `forEach` only for printing.
- Printing one course is done by an **extension function** that you write yourself.
- Write the course type label (M or E) so that, if a new course type is added later, the compiler itself shows that this place needs to be updated.
- The `!!` operator is not allowed.

Expected output

Your program must print this (round averages to two decimals):

```
=== MY GRADE AVERAGE ===
Passed: 6 / 10
Average: 8.33
ECTS-weighted average: 8.18
ECTS earned: 38
Not passed: Physics 1, Mathematics 2, Object-Oriented Programming, Clinical Engineering
Best grade: Fundamentals of Computing (10)

Semester 3:
  Object-Oriented Programming (6 ECTS, M) - not passed
  Medical Image Processing and Analysis (6 ECTS, E) - 9
  Clinical Engineering (6 ECTS, E) - not passed
```

Work and submission rules

- **No AI.** Do not use ChatGPT or similar tools, and turn off AI completion in your editor. You write the code in IntelliJ IDEA; how to set it up and turn off AI is described in `INSTRUCTIONS.md`.
- **Submission via Classmoji**, in the repository you receive (steps in `INSTRUCTIONS.md`). You need **at least 5 commits** that follow your progress, e.g. model, data, queries, output. One big commit at the end leads to fewer points.
- **Live defense in the lab, without AI.** Justify your model, explain your code, then do a small additional task in your own code.

Grading

- The homework is passed if all model requirements and code rules are met, the output is correct and the defense is successful.
- **Fewer points for:** incorrect output, a model that does not meet the requirements, breaking the code rules, a weak defense.
- **If at the defense the student cannot show that they wrote the code themselves, the homework is worth 0 points.**